NAMO

Volunteer Management Platform

Product Specification — MVP

Document Status	Draft
Version	1.0
Date	23 February 2026
Owner	Antigravity / Namo Association
Language	English
Status	Pending stakeholder approval

Table of Contents

Table of Contents 2
1 Executive Summary 4
2 Context & Objective 4
2.1 Current State (As-Is) 4
2.2 Desired State (To-Be) 4
2.3 Success Metrics 4
2.4 Non-Goals (explicitly out of scope) 5
3 Target Users & Personas 6
3.1 User Roles 6
3.2 User Personas 6
Persona A — The Active Clown Volunteer 6
Persona B — The Association Coordinator (Super Admin) 6
Persona C — The External Participant 6
4 Scope 7
4.1 MVP In-Scope 7
4.2 V2+ (Future, Not Committed) 7
5 User Roles & Permissions Matrix 8
6 Core User Journeys 9
6.1 New Volunteer Registration 9
Happy Path 9
Unhappy Paths 9
6.2 External User Registers for an Open Event 9
Happy Path 9
Unhappy Paths 9
6.3 Volunteer Registers for an Event (Happy + Waitlist) 10
Happy Path — Capacity Available 10
Unhappy Path — Event Full, Joins Waitlist 10
Unhappy Path — Overlap Warning 10
Unhappy Path — Tag Not Met 10
6.4 Waitlist Promotion Flow 10
When a Confirmed Volunteer Cancels 10
6.5 Admin Creates and Publishes an Event 11
Admin Bulk-Clones an Event 11
7 Functional Requirements 12
8 Non-Functional Requirements 18
9 Data Model & Permissions 19
9.1 Core Entities 19
9.2 Row-Level Security Summary 20

10 API Contract (Lightweight) 21
Error Shape (all custom endpoints) 21
11 Recommended Tech Stack 22
11.1 Primary Recommendation 22
11.2 Claude Code Optimisation Notes 23
11.3 Environment Variables (Required) 23
12 Architecture Decision Records (ADRs) 24
ADR-01 Auth Strategy: Supabase Auth with Email+Password and Google OAuth 24
ADR-02 Row-Level Security as Primary Access Control Layer 24
ADR-03 Trust Model for Attendance (Passive, No Check-In) 24
ADR-04 Late Cancellation: Configurable Deadline per Event, No Block 25
ADR-05 External Users: Just-in-Time Registration, No Persistent Account 25
ADR-06 Recurring Events: Clone-Based (Not Pattern-Based) 25
13 UX & UI Notes 27
13.1 Primary Screens 27
13.2 Critical UI States 27
13.3 Responsive Design 28
14 Deployment & Operations 29
15 Risks & Rabbit Holes 30
16 Definition of Done 31
Engineering DoD 31
Product DoD 31
Security DoD 31
17 Task Breakdown (Epics & Milestones) 32
Appendix A — Glossary 33
Appendix B — Open Questions Log 33

1 Executive Summary

Namo is a volunteer association operating across multiple sectors: Clown Therapy (hospital visits), school educational laboratories, Compagno Adulto (adult companionship programme), coordination meetings, and special fundraising events.

Today the association manages approximately 50–100 active volunteers through a fragmented combination of Google Calendar invites and manually maintained Excel spreadsheets. This creates significant administrative overhead, scheduling conflicts, no-show tracking gaps, and scaling risk as the association grows.

This document specifies the MVP of a centralised web application — the Namo Volunteer Management Platform — that replaces these manual processes with automated registration, waitlist management, attendance tracking, and email notifications. The platform serves two distinct user populations: internal volunteers (registered members) and external users (occasional participants and partners).

> **Problem in one sentence:** Coordinators spend hours each week manually managing sign-ups, cancellations, and no-shows via Google Calendar and Excel — a process that does not scale and obscures participation data.
> **Solution:** A responsive web app where volunteers self-manage their registrations, admins have full visibility and control, and the system automates waitlist promotion and email notifications.

2 Context & Objective

2.1 Current State (As-Is)

- Event creation: Google Calendar invites sent manually by coordinators.
- Sign-ups: Volunteers accept/decline calendar invites; no waitlist logic.
- Attendance: Tracked manually in Excel after each event.
- External participants: Admin sends individual calendar invites on request; no self-service.
- Clown Therapy presence counter: Maintained manually in a separate spreadsheet.

2.2 Desired State (To-Be)

- Volunteers log in and self-register to events from a central calendar.
- Waitlists are managed automatically (FIFO) with email promotions.
- Attendance is recorded passively (trust model) with admin override.
- External users can browse and register for open events without a pre-existing account.
- Admins have full audit trail, data export, and attendance dashboards.

2.3 Success Metrics

Metric	Target
Admin time spent on scheduling	Reduced by ≥70% vs current (validated at 3-month review)
No-show rate tracking	100% of no-shows recorded in system (vs 0% today)

Waitlist promotion manual interventions	<5% of promotions require admin action
Volunteer self-service registration rate	≥90% of registrations done without admin help
Time to first confirmed registration (new volunteer)	≤10 minutes from account request to first sign-up

2.4 Non-Goals (explicitly out of scope)

The following are plausible but explicitly out of scope for the MVP:

- Sector Admin role (to be decided after MVP launch; defer to V2).
- Compagno Adulto hour tracking (deferred to V2 per client decision).
- Mass communications / newsletter via the platform (use Mailchimp externally).
- Push notifications or SMS (email only for MVP).
- Offline functionality.
- iCal subscription feed (live-updating); MVP supports only downloadable .ics file.
- Automated PDF/Excel reporting; MVP supports raw data export only.
- Payment processing or membership fee collection.
- Multi-language support (Italian UI only for MVP).

3 Target Users & Personas

3.1 User Roles

The MVP supports exactly three roles:

Role	Who they are	Core need
Super Admin (3–5 users)	Association coordinators who manage the platform	Full control: create/edit/delete events, manage users, view all data, export reports, correct attendance.
Volontario (50–200 users)	Registered internal association members	Browse all events, self-register, see who else is going, track personal attendance history.
Utente Esterno (variable)	Occasional external participants; no pre-existing account	Browse events open to externals; provide minimal info to register for a specific event.

3.2 User Personas

Persona A — The Active Clown Volunteer

- Uses the app primarily on mobile, often between commutes.
- Needs to check upcoming hospital visits, see who else is on shift, and quickly cancel if sick.
- Values: speed of sign-up, clear confirmation, no-effort cancellation.

Persona B — The Association Coordinator (Super Admin)

- Creates and manages the weekly event schedule for multiple sectors.
- Needs to override capacity for special events, correct no-shows, and export monthly attendance for the social balance sheet.
- Values: full control, auditability, low maintenance.

Persona C — The External Participant

- Sees a link to an open event (school laboratory) shared via social media or email.
- Has no account; wants to register with minimal friction.
- Values: no signup wall, clear confirmation, easy cancellation.

4 Scope

4.1 MVP In-Scope

Feature Area	MVP Scope
Authentication	Email+password and Google OAuth for internal volunteers; just-in-time info collection for external users.
User Management	Volunteer self-registration (admin-approved), tags/qualifications, account suspension, GDPR data deletion.
Event Management	Full event CRUD: two event types (Interno/Aperto), draft/publish states, location, capacity, min volunteers, required tags, notes, file attachment, cancellation deadline per event, clone and bulk-clone.
Registration & Waitlist	Immediate confirmation if capacity available; configurable FIFO waitlist with auto-promotion; volunteer can refuse promotion; admin can override order (logged).
Cancellation	Any-time cancellation; configurable late-cancellation threshold per event; no reason required; automatic waitlist trigger.
Attendance Tracking	Trust model (+1 on confirmed non-cancelled); admin override with full audit trail; configurable grace period.
Visibility & Privacy	Participant names shown (full name for internal events, name + initial for open events); phone numbers visible to admins only.
Notifications (email)	7 notification types: confirmation, cancellation, waitlist promotion, event reminder, event modified, new event in sector, account approved.
Volunteer Dashboard	Personal history with attendance breakdown by event type; upcoming shifts; iCal download.
Admin Tools	Audit trail, user management (approve/suspend/delete), data export (Excel/CSV), attendance corrections.
UX	Responsive web app, Italian UI, mobile-first design.

4.2 V2+ (Future, Not Committed)

- Sector Admin role with limited permissions.
- Compagno Adulto hour tracking (precise start/end time per session).
- Automated reporting with PDF export for social balance sheet.
- In-app mass communications (currently handled via Mailchimp).
- Live iCal subscription feed (webcal:// link).
- SMS/push notifications.
- Multi-language support.

5 User Roles & Permissions Matrix

All access control MUST be enforced server-side. Client-side hiding is UI convenience only.

Action	Super Admin	Volontario	Esterno	Notes
Create / edit / delete events	YES	No	No	
Publish / unpublish event	YES	No	No	
View all events (Interno + Aperto)	YES	YES	No	
View Aperto events only	YES	YES	YES	Esterno sees only Aperto events
Register for Interno events	YES	YES	No	Tag/qualification check applies
Register for Aperto events	YES	YES	YES	Esterno: just-in-time info collection
Cancel own registration	YES	YES	YES	Flagged as late if past deadline
Cancel any user registration	YES	No	No	Logged in audit trail
Override event capacity	YES	No	No	Logged as override
View all attendee lists	YES	YES (own events)	No	Interno: full name; Aperto: name+initial
View phone numbers	YES	No	No	
View own attendance history	YES	YES	No	Esterno has no persistent account
View all volunteers' attendance	YES	No	No	
Correct attendance records	YES	No	No	Logged in audit trail
Approve / reject new volunteers	YES	No	No	
Assign tags to volunteers	YES	No	No	
Export all attendance data (CSV)	YES	No	No	
Export own personal data	YES	YES	No	GDPR requirement
Download .ics calendar file	YES	YES	No	Own confirmed events only
View audit trail	YES	No	No	

6 Core User Journeys

Each journey includes the happy path, key unhappy paths, and expected observable outcomes.

6.1 New Volunteer Registration

> Preconditions: User has not registered before. Registration page is accessible without login.

Happy Path

1. User opens registration page and fills in: first name, last name, email, phone, nickname, sectors of interest (multi-select), optional notes.
2. User submits form. System creates account with status=pending. Confirmation message: 'Your registration has been submitted. You will receive an email when your account is approved.'
3. Admin reviews pending accounts, clicks Approve. System sets status=active, sends approval email to volunteer.
4. Volunteer logs in and accesses the platform.

Unhappy Paths

- Email already registered: System shows inline error 'This email is already registered. Try logging in.'
- Required field missing: System shows inline field-level validation error before submission.
- Admin rejects registration: Volunteer receives rejection email. Account remains inactive.
- Volunteer tries to access calendar while pending: System shows 'Your account is pending approval' screen with no event content.

6.2 External User Registers for an Open Event

> Preconditions: Event type=Aperto, event is Published, capacity available. User is not logged in.

Happy Path

5. External user lands on the events page (no login required). They see all Aperto events.
6. User clicks 'Register' on an Aperto event.
7. Modal/form appears: 'Enter your details to register'. Fields: First Name, Last Name, Email, Phone.
8. User submits. System creates an external_registration record (status=confirmed). Confirmation email sent.
9. User receives email with event details and a single-click cancellation link.

Unhappy Paths

- Event is full: After submitting info, system informs 'This event is full. You have not been registered.' (No waitlist for external users in MVP.)

- Same email already registered for this event: System shows 'You are already registered for this event.'
- External user tries to access an Interno event URL directly: System shows the public events list with Aperto events only; Interno events are not accessible.

6.3 Volunteer Registers for an Event (Happy + Waitlist)

> Preconditions: Volunteer is logged in (status=active).

Happy Path — Capacity Available

10. Volunteer opens event calendar, sees upcoming events across all sectors.
11. Volunteer clicks on an event, sees event details: title, date/time, location, sector(s), current registrants (by name), available spots.
12. Volunteer clicks 'Register'. System confirms immediately. Registration status=confirmed.
13. System sends confirmation email. Dashboard shows new shift.

Unhappy Path — Event Full, Joins Waitlist

14. Volunteer clicks 'Register'. System shows 'This event is full. Would you like to join the waitlist? You are position X.'
15. Volunteer clicks 'Join Waitlist'. Registration status=waitlist, position assigned.
16. Volunteer sees their waitlist position on the event page and in their dashboard.

Unhappy Path — Overlap Warning

17. Volunteer clicks 'Register' for an event that overlaps in time with an existing registration.
18. System shows warning: 'You are already registered for [Event Name] at the same time. Are you sure you want to register for both?' Volunteer can proceed or cancel.

Unhappy Path — Tag Not Met

19. Volunteer tries to register for an event requiring a tag they do not have. System shows: 'This event requires qualification: [tag name]. Contact your coordinator to add this qualification to your profile.'

6.4 Waitlist Promotion Flow

When a Confirmed Volunteer Cancels

20. Volunteer A (confirmed) cancels. System sets registration status=cancelled, re-opens spot.
21. System immediately identifies Volunteer B (first in FIFO waitlist). Sends email: 'A spot has opened up for [Event Name]. You have been moved from the waitlist to confirmed. Click here to decline if you can no longer attend.'
22. Volunteer B's status updated to confirmed. Waitlist positions for others shift up.
23. If Volunteer B clicks Decline: status reverted to cancelled (not re-waitlisted), next person on waitlist promoted.
24. If no one is on the waitlist: spot remains open; no notification needed.

- Waitlist promotion always occurs, even minutes before the event. No automatic cutoff time.

6.5 Admin Creates and Publishes an Event

25. Admin navigates to Events > New Event.
26. Admin fills in: Title, Type (Interno/Aperto), Sector(s), Date/Time (start + end), Location (mandatory), Max capacity, Min volunteers (optional, internal events only), Required tags (optional), Notes (optional), File attachment (optional), Cancellation deadline (hours before event), Waitlist limit, Event reminder (hours before), State: Draft or Publish immediately.
27. Admin saves. If Published: event appears on volunteer calendar immediately. If Draft: event is hidden from non-admin users.
28. System sends 'New event in your sector' notification to all volunteers with matching sector preferences.

Admin Bulk-Clones an Event

29. Admin opens an existing event, clicks 'Clone'. All fields except date are pre-filled.
30. Admin selects 'Clone to multiple dates', picks 4 specific dates.
31. System creates 4 new events (Draft). Admin reviews and publishes selectively.

7 Functional Requirements

Priority keywords follow RFC 2119: **MUST** = mandatory; **SHOULD** = default unless noted; **MAY** = optional.

ID	Requirement (verifiable statement)	Priority	Acceptance Criteria
AUTH — Authentication & Account			
AUTH-01	The system MUST support email+password login for internal volunteers.	**MUST**	Given valid credentials, when user submits login form, then user is authenticated and redirected to their dashboard within 3 seconds.
AUTH-02	The system MUST support Google OAuth login for internal volunteers.	**MUST**	Given a volunteer clicks 'Sign in with Google', when they complete Google auth, then they are logged into their Namo account (or prompted to link if first use).
AUTH-03	New volunteer self-registration MUST create an account with status=pending; the volunteer MUST NOT access any protected content until an admin approves.	**MUST**	Given a new registration, when volunteer tries to access the calendar while pending, then system shows an approval-pending screen with no event data.
AUTH-04	The system MUST send an email to the volunteer when their account is approved.	**MUST**	Given admin approves account, then within 60 seconds the volunteer receives an email with login instructions.
AUTH-05	The system MUST provide a self-service password reset via email link.	**MUST**	Given user requests reset, then they receive a time-limited (24h) link; using an expired link shows an error, not a reset form.
AUTH-06	Admin MUST be able to suspend or deactivate a volunteer account.	**MUST**	Given admin suspends account, then volunteer's active sessions are invalidated within 60 seconds and they cannot log in.
AUTH-07	When a volunteer's account is suspended or deactivated, all their future confirmed event registrations MUST be automatically cancelled and admin notified.	**MUST**	Given account suspension, then registrations for future events are cancelled, waitlists triggered, admin receives list of affected events.
USER — User & Profile Management			
USER-01	Volunteer registration form MUST collect: first name, last name, email (unique), phone number, nickname, sectors of interest (multi-select), optional notes/special skills.	**MUST**	Given a submission with all required fields, then account is created. Given missing required field, then inline error shown, submission blocked.
USER-02	Admin MUST be able to assign and remove qualification tags to/from any volunteer.	**MUST**	Given admin assigns tag, then volunteer's profile reflects tag and they can register for tag-restricted events.

USER-03	A volunteer MUST be able to request deletion of their personal data at any time.	**MUST**	Given request, then personal data deleted within 30 days; historical attendance records anonymised (user replaced with 'Deleted User').
USER-04	When a volunteer is deleted: personal data MUST be erased; anonymised attendance history MUST be preserved for reporting integrity.	**MUST**	Given deletion, then user row removed from users table; attendance rows retain event/date but user_id replaced with null/anonymous marker.
EVT — Event Management			
EVT-01	Admin MUST be able to create events with all required fields: title, type (Interno/Aperto), sector(s), start datetime, end datetime, location (mandatory), max capacity, cancellation deadline (hours), waitlist limit, reminder timing (hours), draft/published status. Optional: min volunteers, required tags, notes, file attachment.	**MUST**	Given all required fields provided, then event is saved. Given location is empty on submit, then validation error shown.
EVT-02	Events MUST be one of two types: Interno (visible to logged-in volunteers only) or Aperto (publicly visible, no login required).	**MUST**	Given event type=Interno, when unauthenticated user accesses event URL, then 404 or redirect to public events page. Given type=Aperto, page loads without authentication.
EVT-03	Events MUST have two states: Draft (hidden from non-admins) and Published (visible to eligible users).	**MUST**	Given event is Draft, when volunteer browses calendar, then event does not appear. When admin publishes, event appears immediately.
EVT-04	Admin MUST be able to clone a single event; cloned event MUST copy all fields except date and default to Draft status.	**MUST**	Given admin clones event, then new Draft event appears with identical fields (except date=blank). Admin can set date and publish.
EVT-05	Admin MUST be able to bulk-clone an event to multiple specific dates in one operation.	**MUST**	Given admin selects bulk-clone and picks N dates, then N Draft events are created simultaneously.
EVT-06	When editing an event in a series, admin MUST choose scope: 'Only this event', 'This and future events', or 'All events in series'.	**MUST**	Given admin edits time of one event in a series, dialog asks scope. Choosing 'Only this' modifies 1 event; others unchanged.
EVT-07	Admin MUST be able to cancel a future event; all registered volunteers MUST receive a cancellation notification; event is NOT deleted, only marked cancelled.	**MUST**	Given admin cancels future event, then all confirmed+waitlisted volunteers receive email within 60 seconds. Event appears in admin archive with cancelled status.

EVT-08	Admin MAY fully delete a past event only if the event was cancelled and has zero attendance records.	**MUST**	Given event is cancelled+past+no attendance, admin delete option is enabled. Given any attendance records exist, delete button is disabled with tooltip explanation.
EVT-09	Admin MUST be able to override event capacity (register beyond max). Override MUST be recorded in audit trail.	**MUST**	Given admin adds registrant that exceeds max, then registration succeeds, audit log entry created: 'Capacity override by [admin] on [event] at [datetime]'.
EVT-10	Admin MUST be able to reduce event capacity after registrations exist. System MUST warn admin; no registrants are auto-removed.	**MUST**	Given admin reduces capacity below current registrant count, then warning dialog shown. If admin confirms, capacity reduced, existing registrations unaffected.
REG — Registration & Waitlist			
REG-01	When a logged-in volunteer with required tags registers for an event with available capacity, registration MUST be immediately confirmed (no admin approval step).	**MUST**	Given volunteer registers, capacity > 0, tags met: registration status=confirmed, email sent within 60 seconds.
REG-02	When event is at full capacity and volunteer tries to register, system MUST offer waitlist option.	**MUST**	Given capacity=0, when volunteer clicks Register, then system shows 'Event full — join waitlist?' dialog, not a silent error.
REG-03	Waitlist MUST operate on FIFO order (first to join = first promoted).	**MUST**	Given volunteers A, B, C join waitlist in order, when a spot opens, A is promoted first.
REG-04	Waitlist size MUST be configurable per event by admin. When waitlist is full, system MUST inform volunteer that no spot is available (confirmed or waitlist).	**MUST**	Given waitlist at limit, when next volunteer tries to join, then 'No places available (confirmed or waitlist)' message shown.
REG-05	Volunteer MUST be able to see their exact waitlist position.	**MUST**	Given volunteer is on waitlist, their event registration page shows 'You are #N on the waitlist'.
REG-06	When a confirmed registrant cancels, the first waitlisted volunteer MUST be automatically promoted and notified by email including a single-click refuse link.	**MUST**	Given cancellation triggers promotion, then promoted volunteer receives email within 60 seconds with a tokenised refusal link.
REG-07	If a promoted volunteer declines via refusal link, their status is set to cancelled (not re-added to waitlist) and the next waitlisted volunteer is promoted.	**MUST**	Given volunteer clicks refusal link, then status=cancelled, next in queue promoted immediately.

REG-08	Volunteer MUST be able to join waitlists for multiple events simultaneously.	**MUST**	Given volunteer is on waitlist for 3 events, no system error or restriction occurs.
REG-09	Admin MUST be able to register any volunteer to any event, ignoring capacity limits (logged as override).	**MUST**	Given admin registers volunteer on full event, then registration=confirmed, audit log entry created.
EXT — External User Registration			
EXT-01	Aperto events MUST be visible to unauthenticated users without any login prompt.	**MUST**	Given unauthenticated user navigates to events page, then Aperto events are visible. No login prompt appears until they attempt to register.
EXT-02	When an unauthenticated user clicks Register on an Aperto event, system MUST show a lightweight form collecting: first name, last name, email, phone number.	**MUST**	Given user clicks Register, form appears in-page (modal or inline). Volunteer registration page is NOT shown.
EXT-03	External registration MUST be immediately confirmed if capacity is available. No admin approval required.	**MUST**	Given form submitted with valid data and capacity available, then external_registration created (status=confirmed), confirmation email sent.
EXT-04	External users MUST NOT be able to view or access Interno events by any means.	**MUST**	Given unauthenticated access attempt to Interno event URL, then response is 404 or redirect to public page. No event data exposed in API response.
EXT-05	External users MUST receive a single-click cancellation link in their confirmation email.	**MUST**	Given external user registered, email contains unique tokenised cancellation URL. Clicking it cancels registration without login.
CAN — Cancellation Policy			
CAN-01	Any registered volunteer or external user MUST be able to cancel their registration at any time (no hard block, even after deadline).	**MUST**	Given confirmed or waitlisted registration, cancellation button is always available. System may warn but MUST NOT prevent.
CAN-02	Each event MUST have a configurable cancellation deadline (hours before event start) set by admin at creation.	**MUST**	Given admin sets deadline=24h, when volunteer cancels 23h before event, system flags cancellation as 'late'. When volunteer cancels 25h before, no flag.
CAN-03	A late cancellation MUST be recorded in the system and visible in admin reporting. Volunteer is not penalised by the system, but the record exists.	**MUST**	Given late cancellation, then registration record has cancellation_type='late' and timestamp. This is visible in admin attendance export.
CAN-04	Cancellation MUST NOT require a reason from the volunteer — single-click with confirmation dialog.	**MUST**	Given volunteer clicks Cancel, system shows 'Are you sure?' dialog. No reason field. Confirmation completes cancellation.

CAN-05	Admin MUST be able to cancel any volunteer from any event at any time.	**MUST**	Given admin cancels volunteer B from event X, then registration=cancelled, waitlist triggered, email sent to volunteer B.
ATT — Attendance Tracking			
ATT-01	After an event has passed, all confirmed (non-cancelled) registrants MUST be automatically marked as present (+1 attendance).	**MUST**	Given event end time has passed and volunteer did not cancel, then attendance_status=present and counter incremented.
ATT-02	Admin MUST be able to correct attendance (mark no-show, add late presence) during a configurable grace period after the event.	**MUST**	Given admin changes attendance within grace period, then correction is saved. Given grace period expired, edit option disabled with message.
ATT-03	All admin attendance corrections MUST be recorded in the audit trail: actor, timestamp, event, volunteer, old status, new status.	**MUST**	Given correction, audit_log entry created with all fields. Retrievable via admin audit log view.
ATT-04	Each volunteer's personal dashboard MUST show their attendance history broken down by event type (e.g., Clown Therapy: 5, School Labs: 3).	**MUST**	Given volunteer views their dashboard, each event type listed with count. Clicking shows individual sessions.
ATT-05	A volunteer MUST NOT be able to see another volunteer's attendance counter. Only admins can view all counters.	**MUST**	Given volunteer A views volunteer B's profile, counter not shown. Given admin views any profile, counter visible.
NOT — Notifications			
NOT-01	System MUST send an email notification on: registration confirmation, admin-initiated cancellation, volunteer cancellation, waitlist promotion (with refusal link), event modification, new event in sector, account approved.	**MUST**	Given each trigger event, email sent within 60 seconds to all relevant recipients. Email contains event name, date, location.
NOT-02	Event reminder email MUST be sent at a configurable time before the event start (set per event by admin).	**MUST**	Given reminder_hours=24, then email sent 24 hours before event start. Volunteers who have since cancelled do NOT receive reminder.
NOT-03	Volunteers MUST be able to opt out of informational notifications (new events in sector) but MUST NOT be able to opt out of transactional notifications (confirmation, cancellation, promotion, reminder, event modification).	**MUST**	Given volunteer disables 'new event' notifications, they no longer receive NOT-01 'new event in sector' emails but continue to receive all others.

NOT-04	Volunteers MUST NOT receive reminder emails for events they have cancelled from.	**MUST**	Given volunteer cancelled from event, when reminder job runs, their email is excluded from the send list.
EXP — Data Export & Calendar			
EXP-01	Admin MUST be able to export event attendance + volunteer list as Excel/CSV. Export includes: event name, date, volunteer name, status (present/absent/no-show), sector.	**MUST**	Given admin triggers export for a date range, then .xlsx or .csv file downloaded with correct data. No PII of non-relevant users exposed.
EXP-02	Any volunteer MUST be able to download their own personal data export (name, email, phone, attendance history).	**MUST**	Given volunteer requests export, then .csv file downloaded containing only their own data.
EXP-03	Volunteers MUST be able to download an .ics file of their confirmed upcoming events. File includes event title, date/time, location.	**MUST**	Given volunteer clicks 'Download Calendar (.ics)', then browser downloads a valid RFC 5545 .ics file containing their future confirmed registrations.
ADM — Admin Controls & Audit			
ADM-01	System MUST maintain a full audit trail of admin actions: event CRUD, attendance corrections, user role changes, capacity overrides, waitlist overrides.	**MUST**	Given any admin action, audit_log entry created with: actor_id, action_type, entity_type, entity_id, before_state (JSON), after_state (JSON), created_at.
ADM-02	Audit log entries MUST be immutable. No admin can edit or delete audit log entries.	**MUST**	Given admin attempts DELETE on audit_log via API, then 403 Forbidden returned. No delete/update endpoint exposed.
ADM-03	System MUST show a branded maintenance page when unavailable, not a raw server error.	**SHOULD**	Given deployment or outage, user sees branded page: 'Namo is temporarily unavailable, please try again shortly' — not a 500 stack trace.

8 Non-Functional Requirements

Category	Requirement	Target / Constraint
Performance	Dashboard and calendar pages MUST render initial content within 3 seconds on a standard mobile connection (4G, 50ms latency).	p95 < 3s (measured in Vercel Analytics or equivalent)
Performance	All write operations (register, cancel) MUST return a response within 2 seconds.	p95 < 2s under expected load (<200 concurrent users)
Reliability	System MUST have a target availability of 99.5% per month.	Estimated 3.6h downtime/month acceptable for this org size
Security — Access Control	All access control checks MUST be enforced server-side and at the database layer (Supabase RLS). Client-side checks are UX only.	Zero cases of data exposed via API that should be restricted by role
Security — Passwords	Passwords MUST NOT be stored in plaintext. Supabase Auth handles hashing (bcrypt). System MUST NOT log passwords.	No plaintext passwords in logs, DB, or environment variables
Security — Sessions	Sessions MUST expire after 24 hours of inactivity. Refresh tokens handled by Supabase Auth.	No persistent sessions beyond 30 days
Security — GDPR	System MUST support right-to-erasure within 30 days of request. Must display privacy policy link on registration page.	Documented deletion procedure; anonymisation of historical attendance
Observability	Application errors MUST be logged to a monitoring service (e.g. Sentry). Audit trail covers all admin actions.	No silent failures on write operations
Scalability	System MUST support up to 500 concurrent users without degradation.	Supabase free/pro tier + Vercel covers this range comfortably
Accessibility	UI MUST be usable on Chrome/Safari mobile and Chrome/Firefox desktop.	No native app required; responsive PWA behaviour acceptable
Language	UI MUST be in Italian. Error messages, labels, and notifications in Italian.	No i18n framework required for MVP — hardcoded Italian acceptable

9 Data Model & Permissions

This section describes the core entities, their key fields, and data ownership. Full schema migrations are out of scope for this document; this is the 'enough to build safely' sketch.

9.1 Core Entities

Entity	Key Fields	Ownership & Notes
users	id (UUID), email (unique), first_name, last_name, nickname, role (super_admin | volontario | esterno), status (pending | active | suspended | deactivated), phone_encrypted, sectors_of_interest[], notes, created_at, approved_at, approved_by	Row owned by user. Phone stored encrypted. Super admins can read all rows; volontario can read own row only (enforced via RLS).
user_tags	id, name, description, created_by, created_at	Admin-managed. Referenced by events (required_tags[]) and user_tag_assignments.
user_tag_assignments	user_id, tag_id, assigned_by, assigned_at	Admin creates. Volunteer can read own assignments.
events	id (UUID), title, type (interno|aperto), status (draft|published|cancelled|archived), sectors[], start_at, end_at, location, capacity, min_volunteers, required_tags[], notes, file_url, cancellation_deadline_hours, waitlist_limit, reminder_hours, clone_series_id, created_by, created_at, updated_at	Row owned by creator admin. Volontari can read published rows. Esterno can read published+aperto rows only.
registrations	id, event_id, user_id (FK users), status (confirmed|waitlist|cancelled), waitlist_position, registered_at, cancelled_at, cancellation_type (normal|late), is_admin_override, attendance_status (present|absent|no_show), attendance_corrected_by, attendance_corrected_at	Row owned by user. User can read own rows. Admin can read all. Status transitions enforced server-side.
external_registrations	id, event_id, first_name, last_name, email, phone_encrypted, status (confirmed|cancelled), cancel_token (UUID), registered_at, cancelled_at	No user account. Cancelled via tokenised link. PII deletable per GDPR request.
audit_log	id, actor_id (FK users), action_type, entity_type, entity_id, before_state (JSONB), after_state (JSONB), created_at	Append-only. No UPDATE or DELETE permitted. Readable by super_admin only.

notification_preferences	user_id (FK users), informational_emails_enabled (bool, default true)	One row per user. User can update own row.

9.2 Row-Level Security Summary

All RLS policies enforced in Supabase. Key rules:

- **users table:** SELECT own row only for volontario; SELECT all for super_admin; phone_encrypted never returned to non-admin.
- **events table:** SELECT WHERE status='published' AND (type='aperto' OR auth.uid IS NOT NULL) for volontario/esterno; all rows for admin.
- **registrations table:** SELECT/INSERT/UPDATE own rows for volontario; no DELETE; all rows for admin.
- **audit_log table:** INSERT allowed for all authenticated; SELECT/UPDATE/DELETE for super_admin only (no UPDATE/DELETE in practice).

10 API Contract (Lightweight)

The MVP uses Supabase auto-generated REST + Realtime APIs plus custom Edge Functions for business logic. This section defines the primary custom endpoints and their contracts.

Endpoint	Method	Auth	Purpose	Response Shape
POST /api/register-volunteer	POST	None	Submit new volunteer registration	{ id, status:'pending' } | { error }
POST /api/events/:id/register	POST	JWT (volontario)	Register for an event	{ registration_id, status:'confirmed'|'waitlist', position } | { error }
DELETE /api/registrations/:id	DELETE	JWT (owner | admin)	Cancel a registration	{ cancelled_at, is_late } | { error }
POST /api/events/:id/external-register	POST	None	External user registers for Aperto event	{ id, status:'confirmed', cancel_token } | { error }
GET /api/registrations/:cancel_token/cancel	GET	None (token-based)	External user cancels via email link	Redirect to confirmation page
POST /api/events/:id/clone	POST	JWT (admin)	Clone single or bulk dates	{ created_ids:[] } | { error }
GET /api/volunteers/:id/export-ics	GET	JWT (owner | admin)	Download .ics of confirmed future events	Binary .ics file
GET /api/admin/export-attendance	GET	JWT (admin)	Download attendance CSV for date range	Binary .csv file
POST /api/admin/attendance-correction	POST	JWT (admin)	Override attendance status	{ updated, audit_id } | { error }

Error Shape (all custom endpoints)

```
{ "error": "HUMAN_READABLE_CODE", "message": "Italian user-facing message", "details": {}
}
```

11 Recommended Tech Stack

This stack is selected to maximise Claude Code agent productivity, minimise maintenance overhead, and leverage Supabase's built-in capabilities (requested by the client).

11.1 Primary Recommendation

Layer	Recommended Choice	Why it fits	Alternative
Frontend	Next.js 14+ (App Router, TypeScript)	First-class Supabase integration, server components reduce API round-trips, Vercel deployment is 1-click, excellent Claude Code support.	Remix (similar DX); SvelteKit (faster runtime, smaller bundle)
Database & Auth	Supabase (PostgreSQL + RLS + Auth + Storage + Realtime)	Provides RLS row-level security, built-in auth (email + Google OAuth), file storage for event attachments, realtime for live waitlist updates.	PlanetScale + NextAuth (more config required); Neon + Clerk
ORM / Query	Drizzle ORM	Type-safe, lightweight, generates clean SQL, plays well with Supabase.	Prisma (heavier but more mature); raw Supabase client (no type safety)
Email	Resend + React Email	Developer-friendly, generous free tier, supports HTML templates in React.	SendGrid; Nodemailer + SMTP
Hosting	Vercel	Zero-config Next.js deployment, preview URLs per PR, edge middleware support.	Railway (simpler, cheaper at scale); Fly.io
Background Jobs	Supabase Edge Functions + pg_cron	Scheduled jobs (attendance auto-mark, reminders) run inside Supabase with no additional infra.	Inngest; Trigger.dev
Monitoring	Sentry (error tracking) + Vercel Analytics	Free tiers cover MVP scale. Sentry captures unhandled errors; Vercel Analytics provides web vitals.	PostHog; Datadog (overkill for MVP)
Testing	Vitest (unit) + Playwright (e2e)	Both work natively with Next.js + TypeScript. Claude	Jest + Cypress (older, heavier)

		Code can generate and run tests autonomously.	

11.2 Claude Code Optimisation Notes

The following practices maximise Claude Code's ability to build, debug, and extend this codebase autonomously:

- **CLAUDE.md at repo root:** Document all conventions, data model decisions, and environment variable names so Claude Code has persistent context across sessions.
- **Drizzle schema as single source of truth:** Keep all table definitions in /db/schema.ts — Claude Code can read schema to understand data relationships without browsing migrations.
- **Typed API layer:** Use TypeScript throughout. Claude Code produces fewer bugs when types prevent impossible states.
- **Supabase RLS as safety net:** Even if Claude Code writes a server action that forgets an auth check, Supabase RLS prevents data leakage at the database layer.
- **Feature-based folder structure:** /features/events, /features/registrations, /features/users — Claude Code navigates feature folders better than flat structures.

11.3 Environment Variables (Required)

Secrets MUST NOT be committed to the repository. Use .env.local for development and Vercel/Supabase dashboard for production secrets.

12 Architecture Decision Records (ADRs)

ADRs capture significant choices to prevent untracked pivots. Each ADR is either Accepted (in force) or Superseded (replaced by a later ADR).

ADR-01 Auth Strategy: Supabase Auth with Email+Password and Google OAuth

Field	Detail
Status	**Accepted**
Decision	Use Supabase Auth for all authentication. Support email+password and Google OAuth. External users use no auth — just-in-time info collection with tokenised cancellation links.
Rationale	Supabase Auth handles session management, password hashing, OAuth flows, and email verification out of the box. This eliminates ~80% of auth boilerplate. External user just-in-time registration avoids the friction of a sign-up wall for occasional participants.
Consequences	Cannot easily add SMS/magic link later without migrating auth provider (but Supabase supports both). External registration data is stored separately from authenticated users, requiring careful GDPR handling.

ADR-02 Row-Level Security as Primary Access Control Layer

Field	Detail
Status	**Accepted**
Decision	Implement Supabase RLS policies as the authoritative access control layer. All server-side code MUST also check permissions, but RLS is the defence-in-depth.
Rationale	RLS prevents data leakage even if application code has a bug (misused queries, Claude Code generations that miss auth checks). This is the Supabase-recommended pattern and directly addresses OWASP #1 (Broken Access Control).
Consequences	RLS policies must be kept in sync with application logic. Schema migrations that change ownership semantics must also update RLS policies. Developers must be aware that Supabase Dashboard queries bypass RLS by default (use service role carefully).

ADR-03 Trust Model for Attendance (Passive, No Check-In)

Field	Detail
Status	**Accepted**
Decision	Mark confirmed volunteers as present (+1) automatically after event end time. Admin corrects exceptions (no-shows) during a configurable grace period.
Rationale	Association volunteers work with children in hospitals where phone use is prohibited during service. A mandatory check-in would cause data gaps and admin overhead. Trust model is standard for volunteer orgs at this scale.

Consequences	Presence data is only as accurate as admin exception-marking. If coordinators fail to report no-shows, the data drifts. Mitigated by: configurable grace period, admin audit trail, future option to add coordinator reporting in V2.

ADR-04 Late Cancellation: Configurable Deadline per Event, No Block

Field	Detail
Status	**Accepted**
Decision	Each event has a configurable cancellation_deadline_hours set by admin at creation. Cancellations after the deadline are flagged as 'late' in the database but are NOT blocked. Volunteer always has the ability to cancel.
Rationale	Blocking late cancellations causes more harm than good: a blocked volunteer who cannot attend becomes a silent no-show, which is worse than a flagged late cancellation that triggers automatic waitlist promotion. Flexibility per event handles the distinction between rigid internal events and flexible external events.
Consequences	System does not enforce consequences (e.g. account suspension after N late cancellations). This is intentionally deferred — the association will define their own social policy using data from the system.

ADR-05 External Users: Just-in-Time Registration, No Persistent Account

Field	Detail
Status	**Accepted**
Decision	External users (Utenti Esterni) browse Aperto events without an account. When they register, they provide name/email/phone, stored as an external_registration record. They have no login, no dashboard, no persistent profile.
Rationale	Requiring account creation for occasional external participants (e.g., school lab visitors) creates conversion friction and GDPR obligations. Just-in-time is the minimum viable pattern. If external participation grows significantly, V2 can introduce lightweight external accounts.
Consequences	External users have no self-service history. If they want to see their registrations, they must contact admin. Their cancellation is via a tokenised link in the confirmation email. PII is stored and must be handled per GDPR deletion policy.

ADR-06 Recurring Events: Clone-Based (Not Pattern-Based)

Field	Detail
Status	**Accepted**
Decision	Recurring events are managed via a Clone function (single event + bulk-clone to multiple dates), not via a pattern/rule engine (e.g., 'every Tuesday at 14:00').
Rationale	A pattern-based recurrence engine introduces significant complexity (daylight saving time, exceptions, holiday handling, series editing edge cases). For a volunteer org that creates events ad-hoc, the Clone approach provides 90% of the value at 20% of the cost. Matches client preference expressed in survey.

Consequences	Admin must manually clone events rather than setting a recurrence rule. For predictable weekly events this adds minor overhead. A series_id field on events enables grouping cloned events for series-wide edits.

13 UX & UI Notes

These are requirements on externally-observable UI behaviour, not implementation suggestions. Design system and visual aesthetics are outside this spec.

13.1 Primary Screens

Screen	Target User	Key Elements
Event Calendar / Home	All authenticated users (and unauthenticated for Aperto)	Upcoming events as a list or month view; filter by sector; quick register button per event; clear status badge (confirmed / waitlist / open / full).
Event Detail Page	All users	Event info (title, date, time, location, sector, notes); attendee list (with privacy rules applied); registration CTA or status; countdown to cancellation deadline.
My Dashboard	Volontario	Next upcoming shift (most prominent); attendance breakdown by type (chart or counters); past events list; .ics download button.
Admin — Event Management	Super Admin	Table of all events with status; quick publish/unpublish; link to registrations per event; clone button.
Admin — Registrations View	Super Admin	For a selected event: confirmed list, waitlist (with positions), cancelled list; add/remove registrant; override capacity; export CSV.
Admin — User Management	Super Admin	Pending approvals queue (top of page); all users table; assign tags; view attendance; suspend/delete account.
Admin — Audit Log	Super Admin	Reverse-chronological log of all admin actions; filterable by actor, action type, date range.

13.2 Critical UI States

- **Empty state — no events:** Show encouraging message, not blank page. 'No events scheduled yet. Check back soon!'
- **Event full:** Clear visual indicator on the event card (e.g. 'Full — Join Waitlist'). Never a silent failure.
- **Registration loading:** Disable Register button during submission to prevent double-registration. Show spinner.
- **Late cancellation warning:** When volunteer cancels after deadline, show explicit warning: 'You are cancelling after the deadline for this event. This will be recorded as a late cancellation.' Require confirmation.
- **Overlap warning:** Non-blocking. Show warning, allow volunteer to proceed. Do not block the registration.

- **Permissions denied state:** Attempting to access a page without required role shows: 'You do not have permission to view this page.' Not a blank page or 500 error.
- **Network error on write:** 'Something went wrong. Your data has not been lost — please try again.' Do not silently discard the form.

13.3 Responsive Design

- Mobile-first. The primary use case is a volunteer checking their schedule on their phone between activities.
- All core volunteer flows (browse events, register, cancel, view dashboard) MUST be fully functional on a 375px wide viewport.
- Admin tools MAY be desktop-optimised (min 768px) where table-heavy interfaces are required.
- No native app — web app accessible via browser on both iOS and Android.

14 Deployment & Operations

Topic	Specification
Environments	local (developer machine), preview (Vercel PR deployments — one per PR), production (Vercel main branch + Supabase production project).
Secrets handling	Secrets MUST NOT be committed to git. Use .env.local for local dev; Vercel dashboard for preview/production. SUPABASE_SERVICE_ROLE_KEY is server-side only — NEVER exposed as NEXT_PUBLIC_.
Database migrations	Drizzle migrations stored in /db/migrations/. Applied via drizzle-kit push (preview) or explicit migration run (production). Rollbacks planned before every prod migration.
CI/CD	GitHub Actions: on PR — run type-check, lint, unit tests, build. On merge to main — deploy to Vercel production automatically.
Background jobs	Supabase pg_cron for: (1) attendance auto-mark job (runs at event_end_time + 1h); (2) reminder email job (runs hourly, sends reminders for events in window).
Monitoring	Sentry DSN configured in production. Unhandled errors captured with user context (user_id, role). No PII in Sentry breadcrumbs.
Backups	Supabase Pro plan includes automated daily backups with 7-day retention. Enable immediately on production.
Rollout	No feature flags needed for MVP (single deployment). Post-MVP: introduce feature flags for V2 features before enabling for all users.

15 Risks & Rabbit Holes

Risk	Likelihood / Impact	Mitigation
Concurrent registration race condition (two volunteers register for the last spot simultaneously)	Medium / Medium — users perceive double-booking	Use Supabase database transaction with row-level lock on event capacity. Implement optimistic concurrency: check capacity inside transaction, reject second request with 409 Conflict.
Waitlist position drift after admin overrides	Low / Medium — volunteers see incorrect position	Recalculate positions as a computed value (COUNT of waitlisted registrations registered_at < current) rather than storing a mutable integer.
GDPR deletion request affects event integrity	Low / High — deleting a user removes historical data	Implement soft delete: mark user as deleted, anonymise PII fields, retain registration/attendance rows with user_id=NULL.
Email delivery failure (confirmation not received)	Medium / High — volunteer unaware of registration status	Use Resend with delivery webhooks. Log email send attempts in DB with status. Show in-app confirmation immediately after registration (do not rely on email for UX confirmation).
External user registers multiple times (no account de-duplication)	Medium / Low — duplicate records	Check for existing confirmed external_registration by (event_id + email) before creating new record. Return 'already registered' message.
iCal file includes cancelled/past events	Low / Low — confusing calendar	Filter: only confirmed + future events. Regenerate file on each download (no caching of stale data).
Admin accidentally publishes wrong event draft	Low / Medium — confusing for volunteers	Two-step publish: admin clicks Publish → confirmation modal summarises event details → Confirm. Not reversible without 'Unpublish' action.

16 Definition of Done

A feature is considered DONE when ALL of the following are true:

Engineering DoD

- TypeScript type-check passes (tsc --noEmit).
- ESLint passes with no errors (warnings acceptable).
- Unit tests pass for all business logic functions (Vitest).
- Supabase RLS policies written and tested for the affected tables.
- Database migration (if required) is applied and reversible.
- No secrets committed to git.

Product DoD

- All MUST acceptance criteria in Section 7 pass for the affected requirements.
- Happy path works on Chrome mobile (375px) and Chrome desktop.
- All required UI states are implemented: empty, loading, error, success, permissions-denied.
- Italian UI copy used throughout (no placeholder English strings in production build).
- Email notifications triggered correctly (verified in staging with Resend test mode).

Security DoD

- Access control check performed server-side for every protected endpoint.
- RLS policy prevents unauthorised data access (tested with non-admin Supabase client).
- No PII exposed in API responses to users without access rights.
- Audit log entry created for all admin actions covered by FR-ADM-01.

17 Task Breakdown (Epics & Milestones)

Epic	Milestone	Key Deliverables
E1	M1: Foundation	Project scaffold (Next.js + Supabase), Drizzle schema (all tables), RLS policies, auth flows (email+password, Google OAuth), volunteer registration + approval, basic routing and layout.
E2	M2: Event Management	Event CRUD (all fields), draft/publish, event types (Interno/Aperto), clone + bulk-clone, series editing scope dialog, event calendar view (list + month), event detail page.
E3	M3: Registration & Waitlist	Registration flow (confirmed / waitlist), waitlist FIFO logic, waitlist position display, overlap warning, tag-restriction check, admin registration override, external user just-in-time registration.
E4	M4: Cancellation & Attendance	Cancellation flow (any-time, late-flag, no-reason), admin cancellation, waitlist auto-promotion on cancellation, trust-model attendance auto-mark (pg_cron), admin attendance correction + audit trail.
E5	M5: Notifications	All 7 email notification types (Resend + React Email templates), notification preferences (opt-out informational), reminder job (pg_cron), promotion email with refusal link.
E6	M6: Dashboards & Export	Volunteer personal dashboard (history by type, upcoming shifts), admin user management (approve/suspend/delete), admin audit log view, CSV/Excel export, .ics calendar download.
E7	M7: Polish & Launch	Italian UI copy review, mobile responsiveness pass, empty/error/loading states across all flows, Sentry integration, GDPR data deletion flow, load testing, production deployment.

Appendix A — Glossary

Term	Definition
Volontario	A registered internal member of the Namo association. Has a persistent account (pending or active). Can register for both Interno and Aperto events.
Utente Esterno / External User	An occasional participant with no persistent account. Can view and register for Aperto events only via just-in-time info collection.
Super Admin	An association coordinator with full platform access. Can manage events, users, attendance, and audit logs.
Evento Interno	An event restricted to logged-in Volontari only. Not visible or accessible to unauthenticated users.
Evento Aperto	An event open to the public, including Utenti Esterni. Visible without login.
Confirmed Registration	A registration with status=confirmed. Volunteer is expected to attend. Used for attendance (+1) calculation.
Waitlist Registration	A registration with status=waitlist. Volunteer is queued FIFO for the next available spot.
Late Cancellation	A cancellation submitted after the event's configured cancellation_deadline_hours. Flagged in the system but not blocked.
Trust Model (Attendance)	The assumption that a confirmed volunteer who did not cancel attended the event. Marks attendance=present automatically. Admin corrects exceptions.
Grace Period	A configurable window after an event ends during which admins can correct attendance records. After the window, records become immutable.
Clone	The action of creating a new event by copying all fields from an existing event (except date). New event defaults to Draft status.
Bulk Clone	Creating multiple event copies at once, each with a different date specified by the admin.
Audit Trail	An immutable, append-only log of all admin actions. Each entry records: who did what, when, to which entity, and what changed.
RLS (Row-Level Security)	A Supabase/PostgreSQL feature that enforces data access policies at the database layer, ensuring users can only read/write rows they are permitted to access.
ADR (Architecture Decision Record)	A short document recording a significant architectural decision: the context, the decision made, the rationale, and the consequences.
iCal / .ics	An industry-standard calendar file format (RFC 5545). The .ics file can be imported into Google Calendar, Apple Calendar, or Outlook.
Settore / Sector	An activity area of the association: Clown Terapia, Laboratori Scuole, Compagno Adulto, Riunioni, Eventi Speciali. Events can belong to multiple sectors.

Appendix B — Open Questions Log

#	Question	Owner	Status
1	Should the association's name appear in the app as 'Namo' or the full legal name?	Client	Open
2	Is a Sector Admin role needed in V2? What permissions would it have vs Super Admin?	Client	Deferred to V2
3	What is the exact social policy for late cancellations? (E.g., N late cancellations in M months triggers a warning email)	Client	Deferred — V2 feature
4	Should external users (Utenti Esterni) be able to join a waitlist for Aperto events, or is the MVP 'capacity available only'?	Client	To decide before E3 build
5	What is the onboarding migration plan for existing volunteers? (Bulk invite, manual entry, or self-registration with existing email list?)	Client + Dev	To decide before M1 launch
6	Should the app support sub-event structure (e.g., a training day with multiple time slots)?	Client	Deferred — not in MVP scope
7	Is there a minimum browser version requirement (e.g., must work on iOS 14+)?	Client	Assume modern browsers (last 2 major versions)
8	Compagno Adulto: V2 hour tracking — should it be a simple start/end time per session or a more granular log?	Client	Deferred to V2 design phase